
Secure SHell (SSH)



SSH (Secure SHell, RFC 4251): Goals

- ▷ An application and a secure communication protocol over TCP/IP
 - ♦ Allows the establishment of secure remote sessions
 - Replacing insecure telnet/rlogin sessions
 - ♦ Allows the multiplexing of many data flows over a secure session
 - Secure tunneling

- ▷ Security mechanisms
 - ♦ Confidentiality and integrity control
 - Including data flow hiding
 - ♦ Key distribution
 - Session key
 - ♦ Mutual authentication
 - Server host (or server)
 - Client user

SSH: Exploitation

- ▷ Secure remote logins
 - ◆ Instead of telnet/rlogin
- ▷ Secure remote command execution
 - ◆ Instead of rsh/rcmd/rexec
- ▷ Secure file transfer and backup
 - ◆ Secure FTP / Secure copy
- ▷ Port forwarding and tunneling
 - ◆ For adding security to multiple data flows through a single SSH session

SSH: History

- ▷ Created by Tatu Ylönen
 - ♦ First version released in 1995
 - ♦ Second version released in 1998
 - Not compatible with the previous one
- ▷ Björn Grönvall developed OSSH in 1999
 - ♦ From the last open source release of SSH
- ▷ OpenBSD launched the OpenSSH project (www.openssh.org)
 - ♦ Extending OSSH

SSH: Architecture

▶ Client-server architecture

- ♦ Server usually on port 22
- ♦ Services include login, ftp, file copy and TCP tunneling

SSH: Protocols

▷ Transport Layer Protocol (RFC 4253)

- ♦ **Key distribution**
 - Session key computing with ephemeral DH values
- ♦ **Server authentication**
 - With pre-distributed, non-certified public key
 - Signature of DH values and resulting key
- ♦ **Exchange of secure messages**
 - Compression
 - Encryption
 - Integrity control
 - Individual messages
 - Message sequence

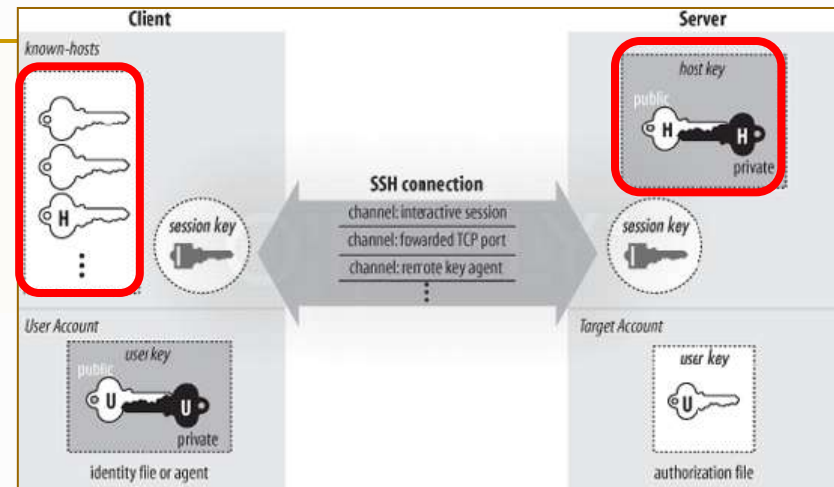
▷ Authentication Protocol (RFC 4252)

- ♦ **User authentication**
- ♦ **Shared key (password), asymmetric keys**

▷ Connection Protocol (RFC 4254)

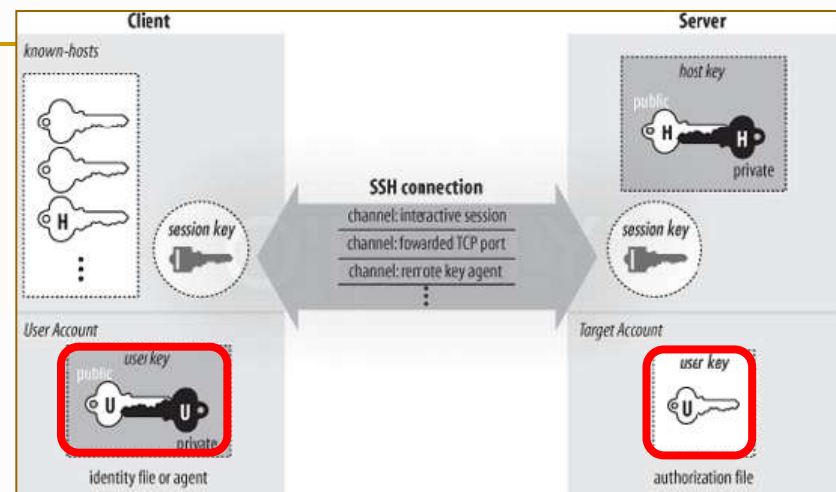
- ♦ **Multiplexing of TCP flows over a single secure session**
- ♦ **Port forwarding**

SSH: Server authn.



- ▷ Each server has an asymmetric key pair
 - ♦ Public key not certified
- ▷ On session setup the server sends its public key
 - ♦ Clients keep a database with known keys
 - IP indexed
 - ♦ Keys are imported when not present
 - Or when different from the current ones
 - Import operations are critical
 - Man-in-the-middle attacks can poison clients' databases with wrong public keys

SSH: Client authn.

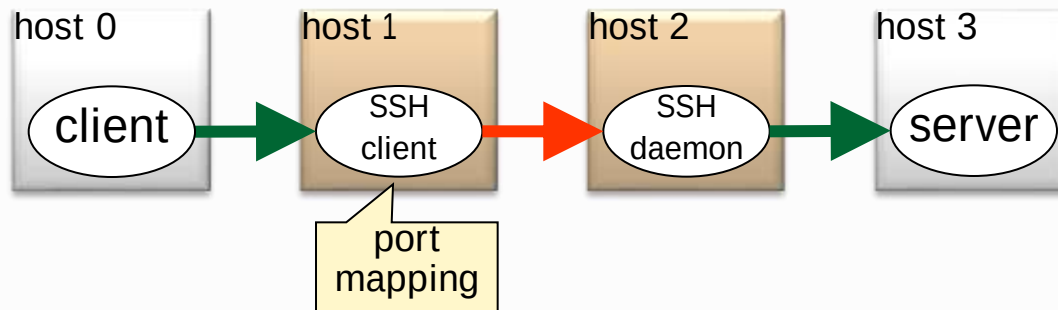


- ▷ Runs over a secure channel
 - ◆ Enables users to exchange passwords with servers for authentication
- ▷ The server publishes the available authentication methods
 - ◆ The user choses the preferred one
- ▷ Authentication methods are defined for users and services
 - ◆ User + service → methods
- ▷ Methods
 - ◆ Shared key (password)
 - The server knows the password (or a transformation of it)
 - ◆ Asymmetric key pair
 - The server knows the public key of the user

SSH: Outgoing (static) tunneling

- ▷ Mapping between an SSH client port and a remote host port

- ♦ Ex. `host1:port1 ⇔ host3:port3`

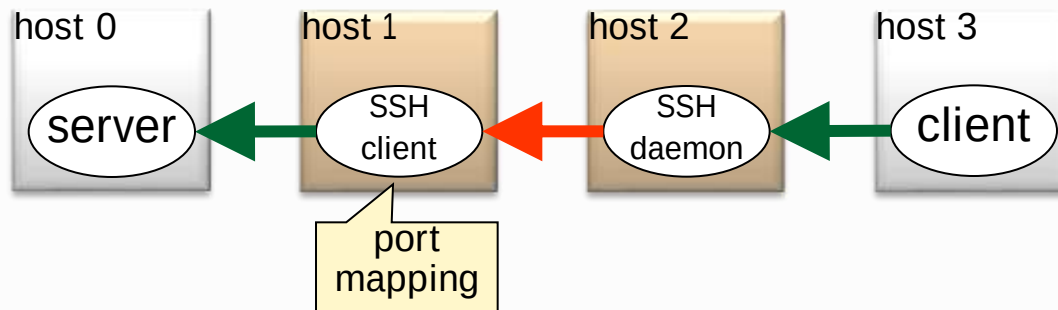


- ▷ The session between host1 and host2 may support TCP connections over a single outgoing tunnel
 - ♦ Many outgoing tunnels may be defined
- ▷ Data flows of all TCP connections over all outgoing tunnels are indistinguishable

SSH:

Ingoing (static) tunneling

- ▷ Mapping between an SSH daemon port and a remote host port
 - ♦ Ex. `host2:port2` ⇔ `host0:port0`

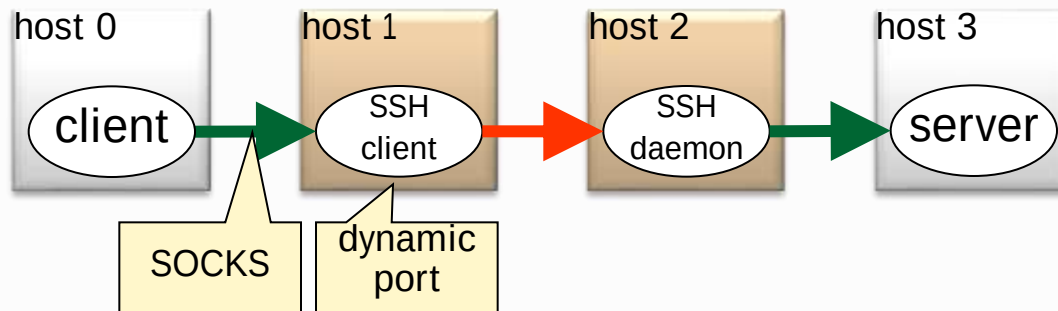


- ▷ The session between host1 and host2 may support TCP connections over a single ingoing tunnel
 - ♦ Many ingoing tunnels may be defined
- ▷ Very useful for remote X11 applications
 - ♦ Running on host3 and using the remote display of host0 / host 1

SSH:

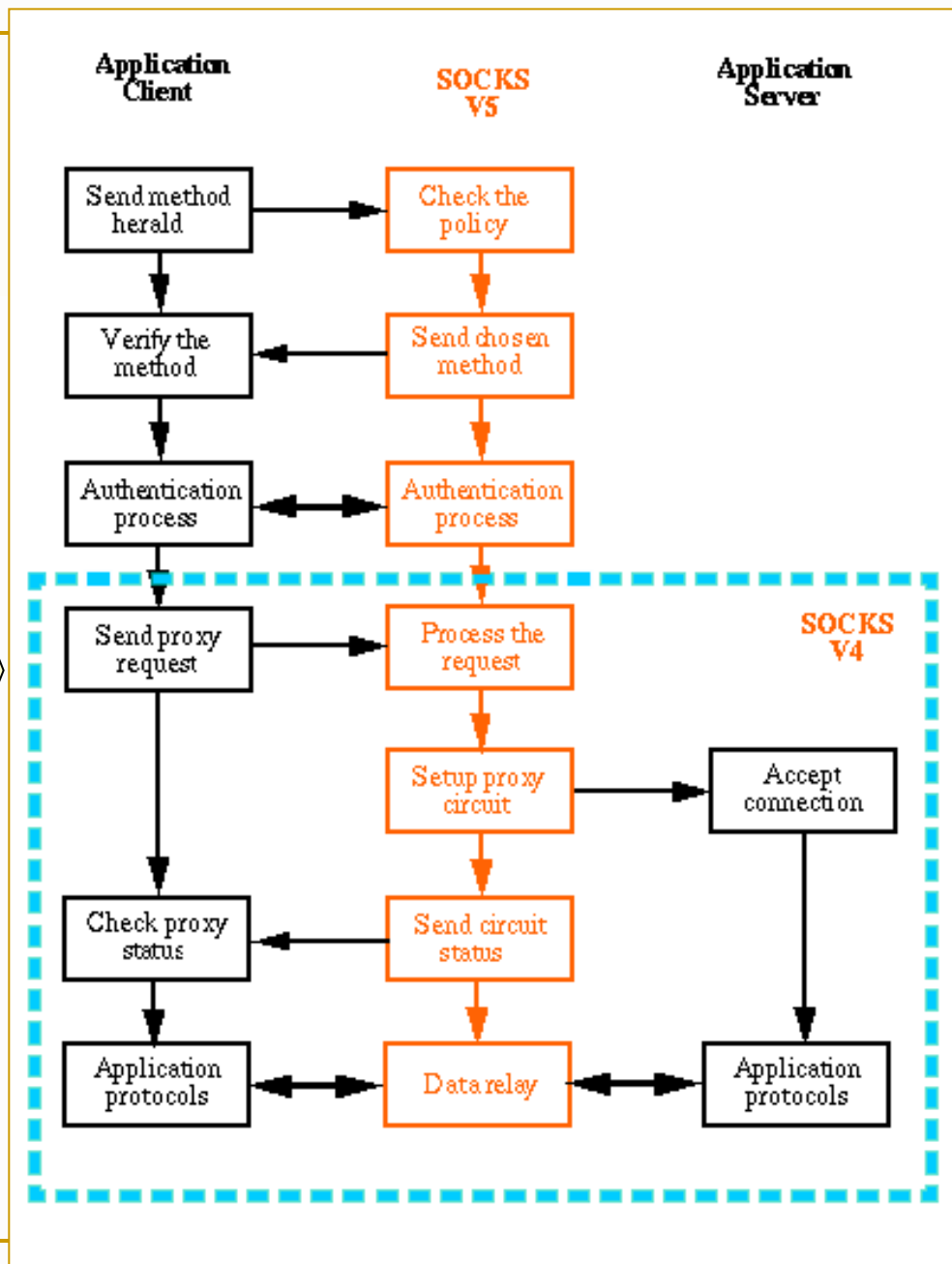
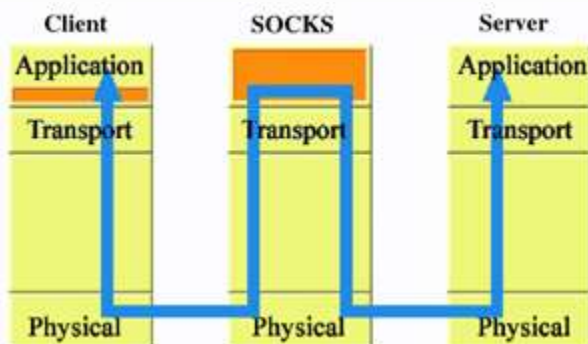
Dynamic (outgoing) tunneling

- ▶ Dynamic mapping between an SSH client port and SOCKS-provided server ports



- ▶ The server end-point address is provided to the SSH client by means of SOCKS
 - ♦ Only works with SOCKS-enable client applications
 - ♦ e.g. browsers

SOCKS (RFC 1928)



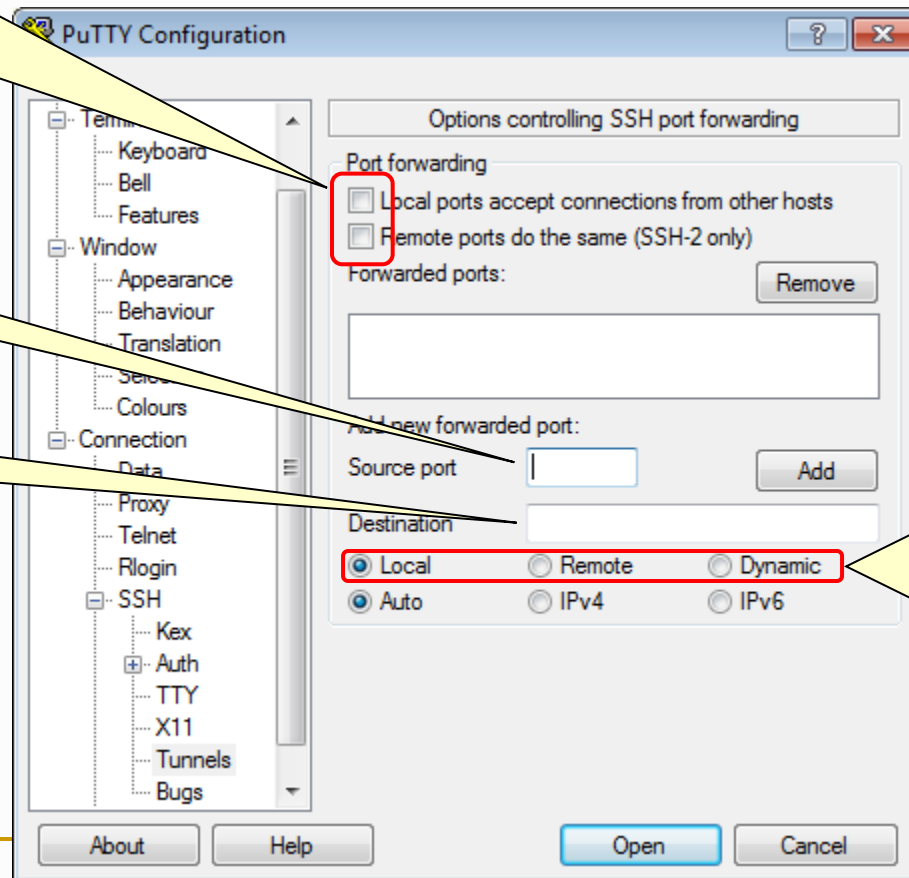
PuTTY SSH client:

Tunnel management (1/2)

acceptance of clients
other than the SSH
client and server hosts

tunnel listening
TCP port

tunnel destination
TCP/IP address

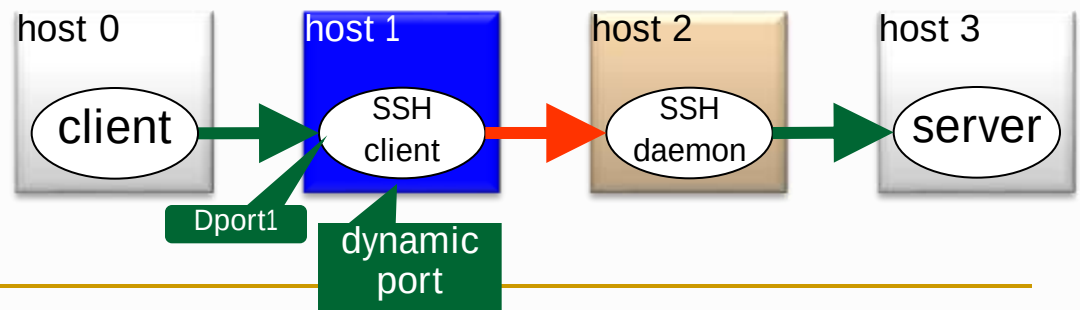
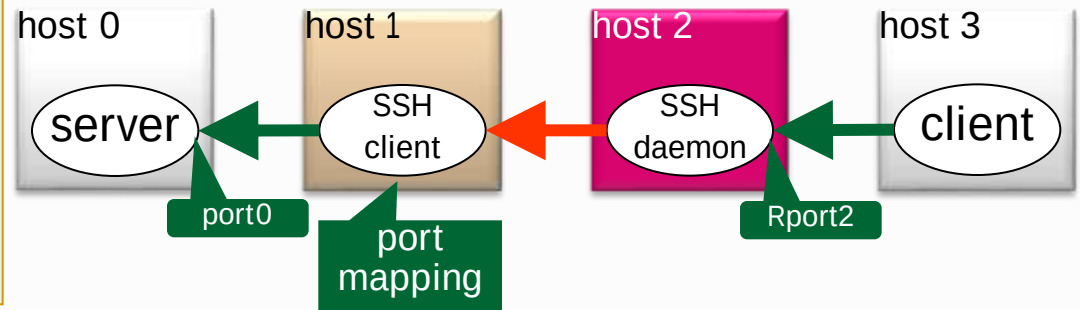
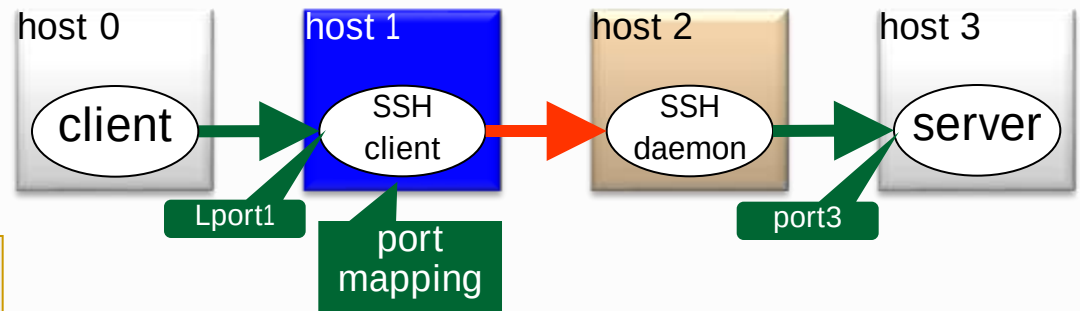
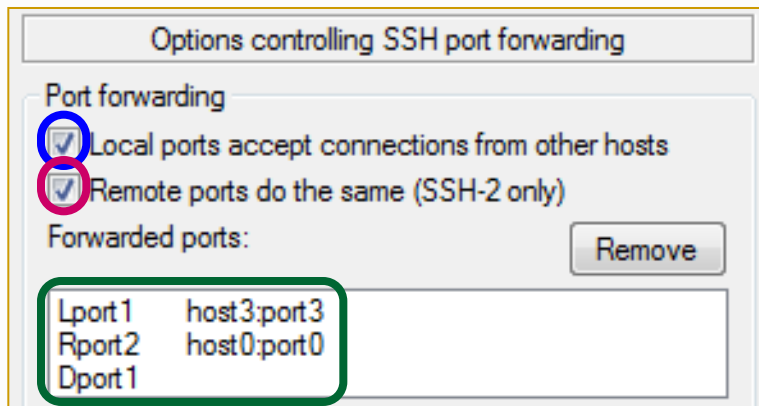


local
(static, outgoing)

remote
(static, ingoing)

dynamic
(local, outgoing)

PuTTY SSH client: Tunnel management (2/2)



SSH agents

- Agents are applications that provide SSH clients a password-free usage of private keys

